

编译器设计专题实验报告

实验四：SLR(1) 分析表生成

张凌曜 2234214014 计算机 2306

1 实验目的

在实验三 LR(0) 规范族的基础上，算 FIRST 集和 FOLLOW 集，生成 SLR(1) 分析表（ACTION 和 GOTO），利用 FOLLOW 集解决 LR(0) 中移进-归约冲突。

2 实验原理

LR(0) 遇到归约项目就无脑归约，SLR(1) 改进了一点：只有当前输入符号在 FOLLOW(A) 里才归约，这样能消掉不少冲突。

FIRST 集就是从某个符号串能推导出来的第一个终结符。FOLLOW 集是在某些句型中能紧跟在非终结符后面的终结符。FOLLOW 集的计算要迭代，直到集合不再变化。

3 实验过程

3.1 FIRST/FOLLOW 集计算

两个集合都用迭代法，反复扫所有产生式直到不动了为止。FOLLOW 集的规则有三条：起始符号的 FOLLOW 加 \$； $A \rightarrow \alpha B \beta$ 时把 FIRST(β) 里非 ϵ 的元素加到 FOLLOW(B)；如果 β 能推出 ϵ 就把 FOLLOW(A) 也加到 FOLLOW(B)。

3.2 分析表构建

对每个状态 I_i 里的每个项目：点后面是终结符 a 就填移进 s_j ；归约项目 $A \rightarrow \alpha \cdot$ 就在 FOLLOW(A) 对应的列填归约 r_k ； $S' \rightarrow S \cdot$ 在 \$ 列填 acc。

4 测试结果

4.1 四则运算文法

FOLLOW 集结果:

$$\begin{aligned} \text{FOLLOW}(E) &= \{\$,), +\} & \text{FOLLOW}(E') &= \{\$\} \\ \text{FOLLOW}(T) &= \{\$,), *, +\} & \text{FOLLOW}(F) &= \{\$,), *, +\} \end{aligned}$$

ACTION 表 (部分):

状态	(	)	*	+	id	\$
I_0	s1				s5	
I_2				s7		acc
I_3			r4	r4	r4	r4
I_4			r2	s8	r2	r2
I_5			r6	r6	r6	r6

I_2 里 $E' \rightarrow E \cdot$ 是归约, $E \rightarrow E \cdot + T$ 是移进。LR(0) 下这是冲突, 但 SLR(1) 里归约只在 $\text{FOLLOW}(E') = \{\$\}$ 时执行, 移进在 $+$ 时执行, 两边不重叠, 冲突解决了。

```

===== SLR(1) 分析表 =====
--- ACTION 表 ---
状态 ( ) * + id $
I0 s1 s5
I2 s7 acc
I3 r4 r4 r4 r4
I4 r2 s8 r2 r2
I5 r6 r6 r6 r6

--- GOTO 表 ---
状态 E F T
I0 2 3 4
I1 6 3 4
I2
I3
I4
I5
I6
I7 3 10
I8 11
I9
I10
I11

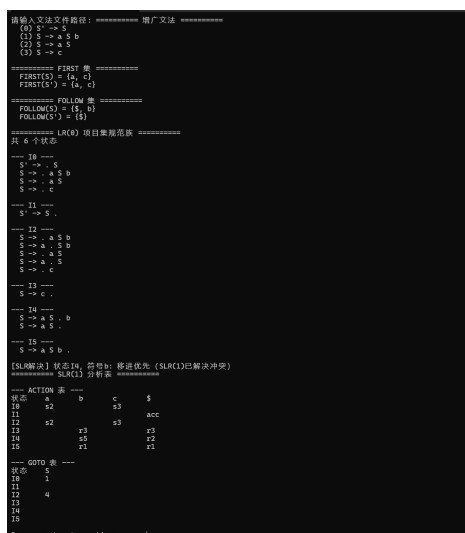
Press any key to continue . . .

```

4.2 嵌套文法

$$S \rightarrow aSb|aS|c$$

I_4 里有 $S \rightarrow aS \cdot b$ (移进) 和 $S \rightarrow aS \cdot$ (归约)。SLR(1) 看 $\text{FOLLOW}(S) = \{\$, b\}$, b 在 FOLLOW 里所以移进和归约对 b 都有定义, 冲突没法完全消掉。程序里对 b 优先移进, 并输出了冲突提示。



5 选做：flex/bison 自动化工具

用 flex 做词法分析、bison 做语法分析，实现了一个四则运算计算器。

5.1 flex 词法定义 (calc.l)

flex 用正则表达式定义 Token 规则，自动生成了扫描器 lex.yy.c:

```

1  [ \t\r\n]+      { /* skip whitespace */ }
2  [0-9]+          { yylval = atoi(yytext); return NUM; }
3  "+"            { return PLUS; }
4  "*"            { return TIMES; }
5  "("            { return LPAREN; }
6  ")"            { return RPAREN; }

```

5.2 bison 语法定义 (calc.y)

bison 用 BNF 定义文法和语义动作，自动生成了 LALR(1) 分析器 calc.tab.c:

```

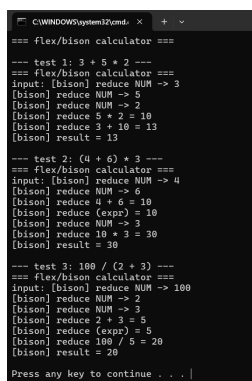
1  %left PLUS MINUS
2  %left TIMES DIVIDE
3
4  expr:
5      NUM                { $$ = $1; }
6      | expr PLUS expr    { $$ = $1 + $3; }
7      | expr TIMES expr   { $$ = $1 * $3; }
8      | LPAREN expr RPAREN { $$ = $2; }
9      ;

```

%left 声明自动解决了运算符优先级和结合性冲突，不需要手动构造分析表。

5.3 测试结果

输入 $3 + 5 * 2$, bison 按优先级先归约 $5 * 2 = 10$, 再归约 $3 + 10 = 13$:



```
C:\WINDOWS\system32\cmd.exe
flex/bison calculator
== test 1: 3 + 5 * 2 ==
flex/bison calculator
input: [bison] reduce NUM -> 3
[bison] reduce NUM -> 5
[bison] reduce NUM -> 2
[bison] reduce 5 * 2 = 10
[bison] reduce 3 + 10 = 13
[bison] result = 13

== test 2: (4 + 6) * 3 ==
flex/bison calculator
input: [bison] reduce NUM -> 4
[bison] reduce NUM -> 6
[bison] reduce 4 + 6 = 10
[bison] reduce (expr) = 10
[bison] reduce NUM -> 3
[bison] reduce 10 * 3 = 30
[bison] result = 30

== test 3: 100 / (2 + 3) ==
flex/bison calculator
input: [bison] reduce NUM -> 100
[bison] reduce NUM -> 2
[bison] reduce NUM -> 3
[bison] reduce 2 + 3 = 5
[bison] reduce (expr) = 5
[bison] reduce 100 / 5 = 20
[bison] result = 20

Press any key to continue . . . |
```

对比手写的 SLR(1) 程序: flex/bison 把词法分析和语法分析的代码生成自动化了,只需要写 .l 和 .y 两个定义文件,就能得到完整的分析器。这就是工业界用 yacc/bison 构建编译器的原因。

6 实验总结

SLR(1) 本质上就是“归约的时候往前看一眼”,用 FOLLOW 集判断该不该归约。大部分常见冲突都能解决,但像嵌套文法那种就不行了,得上 LALR(1) 或者 LR(1)。

FOLLOW 集的迭代计算要注意收敛条件,必须集合完全不变才停,不然会死循环。

选做部分用 flex/bison 体验了一下自动化工具,发现它们和手写 SLR(1) 的核心思路是一样的——都是基于 LALR(1) 分析,只不过 flex/bison 自动帮你做了 FIRST/FOLLOW 集计算和冲突解决。这让人更能体会到理论到工程的桥梁作用。